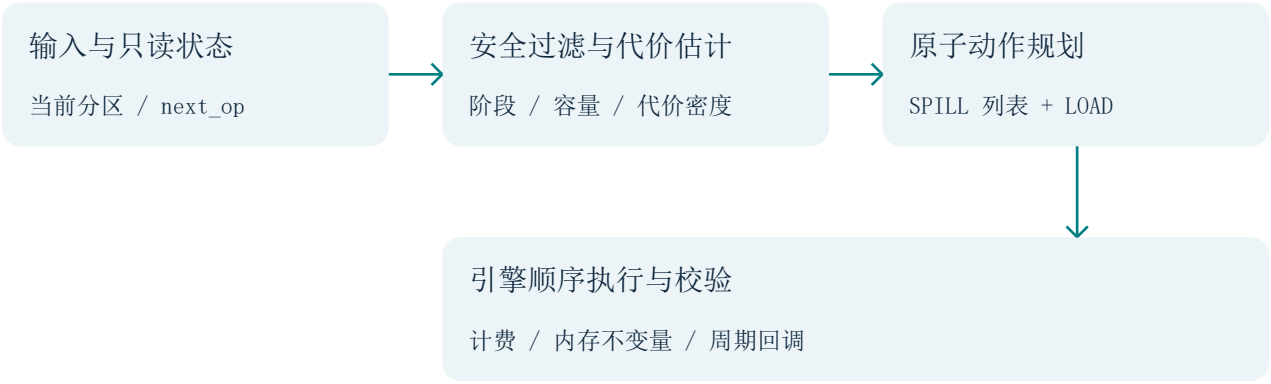
内存受限下 Hash Join Spill-to-Disk 动态调度

安全优先 • 代价感知 • 一步预取

方案名：Cost-Aware Guarded Spill Scheduling
设计日期：2026年10月13日
依据：工作目录全部22张题面截图；已逐张视觉阅读。



目标与交付

在分区原子性与全局内存上限下，完成多算子 Build / Probe 全过程，最小化读写耗时与 Probe 未命中惩罚。提供中文设计文档、C++ 调度器参考实现、可执行验证模型与复现实验数据。

验证边界

算法模型已通过200组合成数据及边界检查；C++ 参考头文件未在本环境编译，尚未进行官方数据集评测。创新点是针对本题的组合设计，不声称学术首创。

题面全景与评分目标

项目	识别结果
系统	K≤8；每算子 P≤64；分区 ID=key%P；总操作数≤5,000,000。
资源	M=64MB至4GB；W、R以字节/微秒计，范围1至10,000。参考硬件为8核CPU、8GB内存、256GB SSD。
操作	INSERT op_id key size；SWITCH op_id；PROBE op_id key。每个算子先插入、再切换、再探测，可跨算子交错。
状态	分区大小与内存驻留标志；算子 BUILD/PROBE 阶段；全局用量；命中、未命中、I/O字节；下一条操作。
调度	内存溢出、分区未命中、每B次有效操作后的周期调整。SWITCH不计入周期。B=1至1,000。
目标	Total_Score=Total_IO_Time+miss_penalty×Miss_Count；越小越好。每次未命中惩罚1至1,000微秒。

题目评判层次

Level 0：功能正确、C++实现、无OOM/崩溃/挂死、格式合规，至少一种优化。Level 1：各组50×BASE/VAL求和；VAL=0或OOM时该组计0。Level 2：数据倾斜及访问模式自适应。Level 3：代价模型、预测预取或跨算子资源分配等创新，并通过对照实验验证。

输出四行

Total_IO_Time: <两位小数>
Hit_Count: <整数>
Miss_Count: <整数>
Total_Score: <两位小数>

评分是模拟I/O成本，不等于程序墙钟运行时间。算法CPU开销不直接出现在该公式中，但仍受最大规模和运行时约束。

输入、操作语义与安全底线

输入契约

第一行 K P M W R miss_penalty；第二行 N B；随后K行 op_id build_count probe_count；再随后N行操作。key为0至10^9，单元组大小1至1,048,576字节。op_id必须按ID定位，不能假定元数据输入顺序就是ID顺序。

计费规则

SPILL：写出整个分区，耗时s/W，驻留改为false，用量减少s。LOAD：读入整个分区，耗时s/R，驻留改为true，用量增加s；只允许PROBE阶段加载。INSERT到磁盘分区：仅追加当前元组，耗时size/W，分区大小增长，全局内存不变。

PROBE进入时分区不驻留即计一次Miss，再释放空间、加载目标；原本驻留才计Hit。空分区仍有状态，按题面状态判断命中，不按键是否实际存在判断。重复落盘仍计全量写成本；不引入“干净页免写”优化。

必须保持的四条不变量

约束	实现检查
容量	0≤memory_used≤M，且等于所有驻留分区大小之和。每一个动作之后检查。
原子性	每个分区全部驻留或全部在磁盘；禁止部分加载、分块探测。
阶段	BUILD分区只可SPILL；LOAD目标必须处于PROBE。
动作顺序	先执行完整SPILL列表，再执行LOAD；Miss回调的最后一个动作必须加载当前目标。

正确性优先于预测：预测错误影响选择质量，不得让容量检查依赖预测。

题面歧义与参考代码风险

发现	处理策略
样例N不一致	截图写N=10，实际有6 INSERT+1 SWITCH+4 PROBE，共11行。纠正为N=11；原框架按N读入会漏最后一条PROBE。
样例文字计数错误	“落盘p1后Hit=2、Miss=1”合计只有3次，不能覆盖4次Probe。保留完整轨迹计算，不能抄该结论。
等大小分区的选择	说明示例先落盘p0；代码按(size,op,pid)降序则选择p1。样例39.92是某一合法轨迹，不是唯一输出。比较必须锁定tie-break。
INSERT目标落盘缺陷	参考processInsert在进入驻留分支后，回调即使落盘目标，也会无条件增加memory_used且不追加磁盘写费。接口兼容方案必须排除当前INSERT目标。
超大分区的可行性	题面未保证每个将被Probe的完整分区≤M。若分区>M且必须整体加载，则任何调度器都无解；不能声称对所有语法合法输入保证无OOM。
可观测性	接口没有每次Hit通知，也没有在State中暴露miss_penalty和build/probe总数。惩罚从构造函数传入；基础方案不读取完整未来轨迹。

框架兼容与扩展版

当前头文件遵守原接口，排除INSERT目标。在可修改引擎的版本中，应在回调后重新读取驻留状态，再决定驻留插入或磁盘追加；同时严格验证LOAD阶段、索引和内存和式。修改引擎可能偏离比赛要求，故作为单独建议，不混入参考调度器。

对当前INSERT目标排除后，若现有目标大小+当前元组大小>M，则该回调无法释放足够空间。这是兼容实现的显式限制，即使之后没有Probe也可能触发。

代价模型：按释放收益选择分区

统一时间量纲

对候选分区i，大小为s_i。定义 C_i=s_i/W+q_i×(s_i/R+penalty)，以及密度 D_i=C_i/s_i。第一项是当前确定写成本，第二项是被淘汰后再次访问的预估损失。优先选择D_i较小的分区。q_i是有界启发式权重，不是经过校准的概率。

可观测信号

为每个分区维护miss-pressure h_i；当前分区Miss后h_i=min(32,h_i+1)，每次周期回调整体乘0.95。q_i=0.15+0.70×h_i/(1+h_i)。若next_op明确探测该分区，q_i置1；BUILD分区再乘0.35，反映短期无需Probe的阶段差异。参数是初始值，需按独立验证集调整。

观测删失必须说明

Miss多不一定等于热度高：驻留分区的命中访问对接口不可见，已落盘的分区更容易积累Miss。这里称为“缺页压力”而非完整访问频率。下一步若允许引擎增加onAccess钩子，再维护每次命中的频率、近期性、按操作时间衰减与访问转移统计。

释放空间是一个离散覆盖问题

设d=need-(M-used)。候选仅取驻留、非空且非当前目标的分区。求集合V使Σs_i≥d，并尽量降低ΣC_i。基础实现使用密度排序后的最短可行前缀，再比较所有单个足够大的分区，最后去掉冗余候选。该算法可行、确定，但不保证全局最优。

为什么增加单分区比较

若缺口10字节，候选A为9字节/成本0.9，B为1,000字节/成本110，C为10字节/成本2，单纯密度会选A+C、成本2.9。单分区比较可改选C、成本2，避免贪心前缀的过度释放。

三个回调如何实现

接口	具体步骤
onMemoryOverflow	need=tuple_size；排除当前INSERT分区；依据安全候选集规划SPILL；返回前保证释放量≥need-(M-used)。只返回SPILL。
onPartitionMiss	先增加当前目标的缺页压力；need=目标完整大小；规划SPILL后追加LOAD目标；禁止选择目标作为victim。
onPeriodicAdjust	衰减缺页压力；只检查next_op；若下一条是PROBE且其目标在PROBE阶段、未驻留且大小≤M，再评估可行预取动作。其他情况返回空列表。

快路径与安全路径

容量足够时不遍历候选。周期没有明确下一条Probe时不主动落盘、不做未知未来预取。状态只读，调度器不能私自修改分区或账目；动作由引擎执行。规划时使用本地freed累加，不能从未执行的State反复判断释放是否足够。

接入方法

把CostAwareScheduler.hpp放在题目数据结构和SpillScheduler基类定义之后。main中以 std::make_unique<CostAwareScheduler>(miss_penalty) 替换原朴素实例。该实现使用原Action构造器、Phase枚举和State字段；要求operators按op_id顺序储存。

实现状态

已交付C++参考头文件及逐步动作一致的Python可执行模型。环境没有可用C++编译器，故尚未验证编译、ABI或官方驱动接入。后续须在官方框架下编译并以同样的样例和边界用例对照。

一步预取与跨算子协作

预取如何真实降低评分

当前操作结束后，在周期回调中已知下一条就是Probe。若现在加载目标，该Probe会计Hit；若等待下一条进入Miss回调才加载，则多付一次penalty。两者对目标的读取字节相同。只有题目以Probe入口驻留状态计数且允许周期LOAD时，这一优势才成立。

接受条件

预取候选大小≤M且属于PROBE阶段；计划先保证容量安全；仅当victim集合的估计损失低于penalty时执行。空闲容量足够时victim成本为0，预取尤其有利。所有预取读取、victim写出和后续再次读写都照常计费。

保守门槛的局限

该门槛不是全局最优判定：若下一条需求加载本来也要淘汰相同victim，那么提早做通常只移动同一组I/O；门槛会错过部分有利预取。它也不能保证未来复用损失小于估计值。以“不预取”消融组对照，才能验证净收益。

跨算子共享，不设固定M/K配额

从全部K×P分区统一选victim，使一个热点算子在其他算子暂时冷却时可多用内存。BUILD阶段给予较低短期保护，PROBE热点给予较高保护。避免硬等分造成一边空闲、一边频繁换入换出。

公平性作为扩展

可增加软惩罚以保护仍活跃算子的最小工作集，或按单位字节得分改善分配；必须保留全局安全兜底，不能强制每个算子留足不可行的固定配额。当前接口无完成通知，不推断算子结束后可以免费释放分区。

正确性、复杂度与大规模实现

容量安全的归纳论证

初始用量为0。SPILL只减少已驻留分区的完整大小；LOAD只在释放后可用容量≥目标大小时执行。INSERT目标被排除，回调后仍驻留，释放量满足tuple_size需求；磁盘INSERT不增加用量。SWITCH不改变用量。故在可行输入假设下，每一步used≤M且账目与驻留和式一致。

动作完整性

候选每个分区只出现一次，故不重复SPILL。Miss最后一次动作LOAD目标；周期预取只LOAD处于PROBE的next目标。若所有候选释放后仍不足，明确返回不可行错误，不能把非法动作或假命中作为兜底。

成本与空间

设Q=K×P≤512。一次压力回调的候选排序O(Q log Q)，单分区比较及冗余删除最坏O(Q²)，空间O(Q)。当前vector逐项erase会产生O(Q²)移动；可用标记筛选降低到O(Q)。周期衰减O(Q)，无需保存N条工作负载。

上限压力不能忽略

N=5,000,000且B=1时，仅全表周期衰减就可能执行约25.6亿次更新；频繁排序和State复制也可能成为瓶颈。当前交付是可审阅参考实现，不宣称已通过最大规模时限。生产实现应使用懒衰减、预分配候选数组和按版本维护的候选顺序。

工程路线

用流式当前/下一条双缓冲读取替代vector<string>保存全部输入。size、memory、I/O字节和计数使用int64_t；时间使用double，仅输出时保留两位小数。避免以四舍五入后的分数比较策略。审计检查用于测试构建，提交构建保留关键边界校验。

对照验证与可复现结果

样例输入修正后的实测模型

策略	I/O微秒	Hit / Miss	Score
截图说明的p0轨迹	9.92	1 / 3	39.92
确定tie-break的朴素模型	7.04	2 / 2	27.04
代价选择，关闭预取	7.04	2 / 2	27.04
代价选择，开启预取	7.04	3 / 1	17.04

前三项区别必须说明：39.92来自截图中的合法示例轨迹；可执行模型的朴素组使用(size,op,pid)降序tie-break，并排除INSERT目标以修复参考框架缺陷。不能把27.04到17.04的收益归因于单独的代价选择；此样例主要验证预取。

200组固定种子合成数据

策略	汇总Score	相对朴素下降
朴素修复模型	426694.710	0.00%
代价模型，无预取	396227.595	7.14%
代价模型，有预取	367944.815	13.77%

随机种子20261013；K为1至4，P为2至8，M为128至512字节，B为1至20；每组最多120次Build尝试和250次Probe，Probe约65%集中于分区0。完整分区大小限制≤M。W=100、R=80、penalty=10。

这些是微型倾斜工作负载，M低于正式范围，主要验证逻辑与消融，不代表500万操作、全速硬件或正式数据分布；汇总Score比不是题面Level1逐组分数之和。

创新点、实验扩展与风险控制

创新点	作用与验证方式
统一代价密度	把写成本、预计读回和未命中惩罚统一为时间/字节；与Largest-First比较。200组模型中的关闭预取组用于隔离此项收益。
贪心覆盖修正	密度前缀、单个足额分区比较、冗余删除，降低小缺口下过度释放。需增加“只用密度前缀”消融；当前未单独量化。
受控一步预取	利用严格许可的next_op；把确定的下一次Miss转换为Hit，并完整记录I/O。已与关闭预取组比较。
阶段与全局协作	同一候选集覆盖全部算子，BUILD/PROBE区别保护。需比较固定配额和取消阶段权重；当前未单独量化。

正式实验矩阵

补齐均匀访问、Zipf式倾斜、热点迁移、周期扫描、单分区超过50%、多算子交错、读写带宽极端、低/高惩罚、B=1/1000，以及N=500万压力。报告每组Score、读写字节、Hit/Miss、动作数、峰值内存、墙钟与P95运行耗时。

避免实验泄漏

参数在独立验证集调节，正式测试集只评测一次。离线全知策略可作为研究参照，但不接入在线实现；文件中可见完整轨迹不等于调度器接口获准使用完整未来。

当前边界测试

验证空分区探测、已落盘分区追加、两个算子BUILD/PROBE交错及B=1预取；所有动作检查驻留和式与容量；每组Hit+Miss=250；构造目标增长到M+1的场景，确认明确识别不可行。

实施路线与验收清单

阶段	工作及完成标准
已交付	22张截图阅读；设计说明；参考C++头文件；Python模型；样例纠错；200组数据与边界检查；PDF渲染检查。
正式框架接入	编译C++17；补足参考框架缺失的<memory>、<tuple>等头文件；锁定官方基线；比对样例每个动作与计数。
性能实现	流式输入、懒衰减、固定容量数组、减少只读State全量复制；跑到K=8、P=64、N=500万。
效果评估	独立测试集和各优化消融；验证带宽/惩罚变化时收益；对热点迁移和扫描退化有可解释记录。
最终提交	正式数据无OOM；输出四行格式正确；C++构建通过；算法与结果来源清晰；不可行输入与框架假设披露完整。

文件与复现

CostAwareScheduler.hpp：调度器参考实现。
verify_design.py：执行算法模型并输出validation_results.json。
sample_corrected.txt：N=11的修正样例。
validation_results.json：逐步样例轨迹及聚合验证结果。
screenshot_inventory.csv：截图索引。
build_report.py：说明书生成源。

运行方式

在项目目录执行 python design/verify_design.py 复验；PDF生成需reportlab，执行 python design/build_report.py。C++构造器接收miss_penalty；false参数可关闭预取进行消融。

引用范围

本方案仅使用提供的题面截图作为规则来源，算法与合成验证在本地推导。截图中列出的论文名称未进行外部核验，不作为论据或声称复现其算法。

附录：全部截图阅读索引

共22张，按时间顺序阅读。以下文件均位于项目根目录；没有发现额外题目附件。

序号	截图文件	识别内容
1	屏幕截图 2026-10-13 101606.png	背景、矛盾与任务
2	屏幕截图 2026-10-13 101627.png	系统参数及数据结构
3	屏幕截图 2026-10-13 101708.png	INSERT/SWITCH/PROBE语义
4	屏幕截图 2026-10-13 101804.png	SPILL/LOAD及约束
5	屏幕截图 2026-10-13 101821.png	三个回调与可见State
6	屏幕截图 2026-10-13 101832.png	Build样例
7	屏幕截图 2026-10-13 101839.png	Probe样例与对比
8	屏幕截图 2026-10-13 101852.png	目标、规模及输入首行
9	屏幕截图 2026-10-13 101903.png	后续输入、约束与输出
10	屏幕截图 2026-10-13 101916.png	完整样例输入输出
11	屏幕截图 2026-10-13 101938.png	参考框架结构体上半
12	屏幕截图 2026-10-13 101948.png	SystemState与调度器基类
13	屏幕截图 2026-10-13 101956.png	朴素溢出回调
14	屏幕截图 2026-10-13 102005.png	朴素Miss/周期回调
15	屏幕截图 2026-10-13 102014.png	引擎字段与State拷贝
16	屏幕截图 2026-10-13 102024.png	动作执行与构造器
17	屏幕截图 2026-10-13 102031.png	算子初始化与INSERT
18	屏幕截图 2026-10-13 102039.png	SWITCH/PROBE与输出
19	屏幕截图 2026-10-13 102047.png	主函数及预读输入
20	屏幕截图 2026-10-13 102056.png	next_op与执行循环
21	屏幕截图 2026-10-13 102105.png	框架说明及参考文献
22	屏幕截图 2026-10-13 102112.png	Level0至Level3评判标准